**Projet de Programmation**
**Master Informatique**
**2025–2026**

université de BORDEAUX

# Carcassonne – Python

## Rapport préliminaire

Anthony BEAUVIEUX, Tony HUANG, Maxence SALIN, Clément SIMON, Loïc THIN

# 1 Présentation du jeu

## 1.1 Contexte

**Carcassonne** est un jeu de stratégie et de placement de tuiles créé par Klaus-Jürgen Wrede en 2000, conçu pour 2 à 5 joueurs, et pouvant accueillir jusqu'à 6 joueurs grâce à l'extension *The River*. Il s'agit d'un jeu stochastique à information parfaite, c'est-à-dire un jeu où le hasard intervient dans le tirage des tuiles mais où l'ensemble des éléments présents sur le plateau est visible de tous les joueurs.



FIGURE 1 – Tuile de départ

La version considérée ici utilise les 72 tuiles du jeu de base, auxquelles s'ajoutent les 12 tuiles de l'extension *The River*. Parmi les 72 tuiles se trouve la tuile de départ (voir figure 1) qui est placée au centre du plateau avant que la partie commence. Cette tuile est la même pour chaque partie comportant une *City* en haut (couleur marron), une *Road* qui traverse de gauche à droite et un *Field* en bas. Chaque joueur dispose de 7 pions de sa couleur pour marquer des points au cours de la partie. Le but du jeu est que les joueurs construisent progressivement le plateau, à tour de rôle, en posant des tuiles de manière cohérente et en plaçant stratégiquement leurs pions pour accumuler des points. Le vainqueur est le joueur qui totalise le plus de points à la fin de la partie.

## 1.2 Règles

Les règles de **Carcassonne** utilisées sont celles de la version 2012[1].

### 1.2.1 Spécificités

**Tuile**

Chaque tuile possède quatre côtés ainsi qu'un centre. Chaque côté correspond à un type de terrain parmi *Road*, *City* ou *Field*. Le centre peut représenter une *City*, un *Field*, une *Banner* ou un *Monastery*.



FIGURE 2 – Exemple d'une tuile avec une *City*, une *Road* et un *Field*

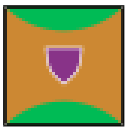


FIGURE 3 – Exemple d'une tuile où une *City* possède une *Banner*

**Pion**

Lorsqu'un pion est placé sur une tuile, il prend un rôle en fonction du type de terrain occupé : *Knight* sur une *City*, *Farmer* sur un *Field*, *Thief* sur une *Road* et *Monk* sur un *Monastery*.

### 1.2.2 Déroulement d'un tour

Les joueurs jouent à tour de rôle selon un ordre prédéfini (généralement dans le sens horaire, en commençant par le joueur le plus jeune). À son tour, un joueur effectue les actions suivantes dans l'ordre indiqué :

1. **Piocher une tuile** Le joueur pioche une tuile dans la pioche actuellement active. Cette tuile est visible par l'ensemble des joueurs.
2. **Placer la tuile** La tuile piochée doit être placée sur le plateau en respectant les règles suivantes :
   - La tuile doit être adjacente à au moins une autre tuile déjà posée.
   - Les côtés en contact doivent être de même type de terrain.
3. **Placer un pion (optionnel)** Le joueur peut placer un pion sur une partie de la tuile qu'il vient de poser (un des quatre côtés ou le centre).
4. **Restriction de placement des pions** Si une partie de terrain de la tuile posée est reliée à une partie du même type déjà occupée par un pion, aucun nouveau pion ne peut y être placé. Les *Road* et *City* agissent comme des séparateurs entre les terrains.
5. **Résolution des terrains complétés** Si la pose de la tuile complète un ou plusieurs terrains, ceux-ci sont résolus immédiatement :

   **Road**
   Une route est complète lorsque ses deux extrémités atteignent une intersection ou une fin de route. Le joueur possédant le plus de pions (*Thief*) sur cette route gagne un nombre de points égal au nombre de tuiles composant la route. En cas d'égalité, tous les joueurs concernés marquent ces points.

   **City**
   Une ville est complète lorsqu'elle est entièrement fermée. Le joueur possédant le plus de pions (*Knight*) gagne 2 points par tuile composant la ville, ainsi que 2 points par *Banner*. En cas d'égalité, les points sont attribués à tous les joueurs concernés.

   **Monastery**
   Un monastère est complet lorsque les 8 cases entourant sa tuile sont occupées. Le joueur possédant le pion dans le *Monastery* (*Monk*) gagne alors 9 points.

   **Field**
   Les champs ne sont résolus qu'à la fin de la partie. Un pion dans un *Field* (*Farmer*) reste donc en jeu jusqu'à la fin.
6. **Retour des pions** Tous les pions associés à des *City*, *Road* ou *Monastery* complétés durant ce tour sont retirés du plateau et rendus à leurs propriétaires.

### 1.2.3 Fin de partie

La partie prend fin lorsqu'il n'y a plus de tuiles dans la pioche principale. Les joueurs ajoutent alors à leur score les points correspondant aux terrains non complétés, selon les règles suivantes :

**Road**
Une route incomplète rapporte 1 point par tuile la composant au joueur possédant le plus de pions (*Thief*) sur cette route. En cas d'égalité, tous les joueurs concernés marquent ces points.

**City**
Une ville incomplète rapporte 1 point par tuile la composant ainsi que 1 point par *Banner* présent, pour le joueur possédant le plus de pions (*Knight*). En cas d'égalité, les points sont attribués à tous les joueurs concernés.

**Monastery**
Un monastère incomplet rapporte 1 point par tuile adjacente, y compris la tuile du monastère, pour un total compris entre 2 et 8 points.

**Field**
Chaque *Farm* (ensemble de *Field* connectés ; un *Field* isolé constitue une *Farm*) est attribuée au joueur possédant le plus de pions (*Farmer*). Ce joueur marque 3 points par *City* complète adjacente à la *Farm*. En cas d'égalité, les points sont attribués à tous les joueurs concernés.

Le joueur ayant obtenu le plus grand nombre de points à l'issue du décompte final remporte la partie.

### 1.2.4 Extension : *The River*

L'extension *The River* introduit 12 tuiles supplémentaires et modifie exclusivement la phase de mise en place du jeu. La tuile de départ standard est remplacée par une tuile *Lake*, qui marque la source de la rivière. Les autres tuiles de l'extension sont mélangées, à l'exception de la seconde tuile *Lake*, placée volontairement en fin de pile afin de constituer l'embouchure de la rivière. Les joueurs piochent ensuite ces tuiles à tour de rôle et les placent conformément aux règles habituelles de Carcassonne, de manière à former un cours d'eau continu. La phase de mise en place se termine lorsque la tuile *Lake* finale est posée, complétant ainsi la rivière. À partir de ce moment, la partie se poursuit selon les règles standard de Carcassonne, sur un terrain initialement structuré.

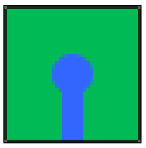


FIGURE 4 – Tuile *Lake* de l'extension *The River*

Afin de garantir la cohérence du terrain et d'éviter toute situation de blocage, il est interdit de placer une tuile de rivière de manière à orienter le cours d'eau vers une autre tuile de la rivière déjà posée.

## 1.3 Complexité

**Carcassonne** est un jeu aux règles simples, mais qui offre une grande richesse stratégique, les choix des joueurs se complexifiant à mesure que le plateau s'étend. Ainsi, sa complexité est comparable à celle de jeux classiques comme les échecs ou le shogi.

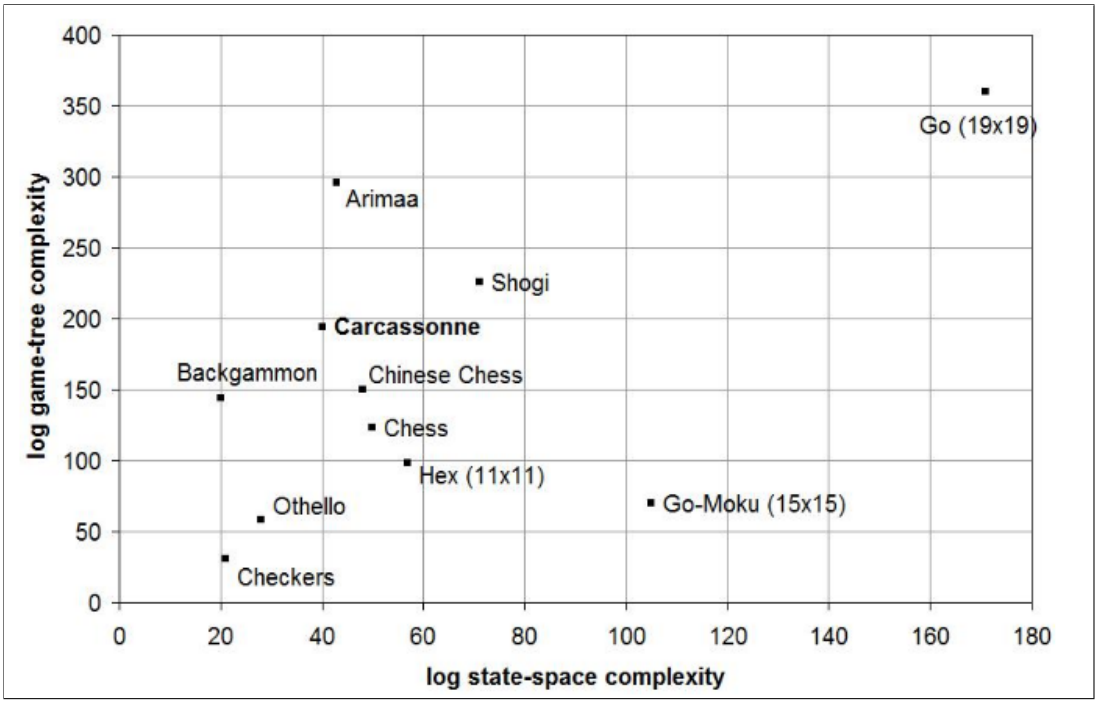



FIGURE 5 – Comparaison de la complexité de Carcassonne avec d'autres jeux [2]

La complexité combinatoire du jeu est particulièrement élevée en raison du grand nombre de tuiles et des multiples façons dont elles peuvent être placées sur le plateau, ainsi que des stratégies variées que les joueurs peuvent adopter selon la configuration des tuiles et la position des pions. La complexité de l'arbre de jeu est estimée à environ $10^{194}$ [2], ce qui rend toute résolution complète du jeu par des méthodes de recherche exhaustive irréaliste. Par conséquent, la création d'une intelligence artificielle capable de jouer parfaitement à **Carcassonne** devient impossible. Le brute-force ne sera alors pas une solution envisageable pour notre projet.

# 2 Algorithmes et structures de données

# 3 Architecture

# 4 Spécifications étendues

| Besoin | Ordre de priorité | Heure de travail estimé |
|---|---|---|
| F1 : Lancement du programme | Essentiel | 5 |
| F2 : Fichier de configuration | Secondaire | 3 |
| F3 : Internationalisation (i18n) | Optionnel | 5 |
| F4 : Interface en ligne de commande | Secondaire | 3 |
| F5 : Interface graphique | Secondaire | 5 |
| F6 : Mode blitz | Optionnel | 2 |
| F7 : Mode contest | Optionnel | 2 |
| F8 : Graine aléatoire | Secondaire | 2 |
| F9 : Mélange des tuiles | Essentiel | 8 |
| F10 : Options spécifiques | Secondaire | 2 |
| F11 : Gestion des règles | Essentiel | 15 |
| F12 : Gestion des erreurs de l'utilisateur | Secondaire | 5 |
| F13 : Gestion du undo/redo | Secondaire | 5 |
| F14 : Gestion des conseils | Optionnel | 1 |
| F15 : Shell interactif | Secondaire | 4 |
| F16 : Proposer de sauvegarder avant de quitter | Optionnel | 1 |
| F17 : Édition de la commande en cours | Secondaire | 4 |
| F18 : Historique des commandes tapées | Secondaire | 4 |
| F19 : Complétion des commandes | Secondaire | 4 |
| F20 : Notation des coups | Essentiel | 5 |
| F21 : Sauvegarde et restauration d'une partie | Secondaire | 5 |
| F22 : Commentaires dans la sauvegarde | Optionnel | 2 |
| F23 : Format des paramètres du jeu | Secondaire | 2 |
| F24 : Format du plateau de jeu | Essentiel | 10 |
| F25 : Format de l'historique | Secondaire | 2 |
| F26 : Module graph | Essentiel | 8 |
| F27 : Algorithmes de manipulation des graphes | Essentiel | 10 |
| F28 : Menus de l'application | Secondaire | 5 |
| F29 : Raccourcis clavier | Optionnel | 2 |
| F30 : Drag'n'Drop | Secondaire | ? |
| F31 : Temps de réflexion borné | Secondaire | ? |
| F32 : Minimax avec alphabeta-pruning | Essentiel | ? |
| F33 : Fonctions d'évaluation pour Minimax | Essentiel | ? |
| F34 : Approfondissement itératif de Minimax | Secondaire | ? |
| F35 : Profondeur de recherche de Minimax | Secondaire | ? |
| F36 : Monte Carlo Tree Search (MCTS) | Secondaire | ? |
| F37 : Fonction de sélection par Machine Learning | Secondaire | ? |
| F38 : Découverte de serveurs et connexion client-serveur basique | Secondaire | ? |
| F39 : Serveur multi-clients et gestion de parties | Secondaire | ? |
| F40 : Système d'invitations et gestion avancée des joueurs | Secondaire | ? |

# Références

[1] Christopher Obe. Carcassonne complete annotated rules. https://www.dover.nh.gov/Assets/government/city-operations/library/borrow/library-of-things/carcassonne-rules.pdf, 2014.

[2] Cathleen Heyden. Implementing a computer player for carcassonne. Master's thesis, Maastricht University, 2009. https ://project.dke.maastrichtuniversity.nl/games/files/msc/MasterThesisCarcassonne.pdf.